Capacités numériques

— NUM —

Les outils numériques occupent aujourd'hui une place centrale en chimie, non pas comme une fin en soi, mais comme un prolongement naturel de la démarche scientifique. En classe préparatoire TPC, ils permettent de simuler des systèmes chimiques, d'exploiter des données expérimentales, de tester des modèles cinétiques ou thermodynamiques, de visualiser des évolutions et surtout de relier plus finement observations et modèles théoriques. Leur maîtrise ne relève donc pas d'une simple compétence informatique : elle fait partie intégrante de la formation du futur scientifique, capable de modéliser un système, de confronter ses prédictions aux données et d'en analyser la validité.

L'apprentissage de l'algorithmie contribue également à structurer la pensée scientifique. Il oblige à décomposer un problème complexe en étapes simples, à expliciter un raisonnement et à anticiper ses différentes issues. Cette rigueur développe des compétences transversales clés : clarté du raisonnement, contrôle des hypothèses et capacité à corriger une démarche. Elle dépasse largement le seul cadre du numérique et renforce la maîtrise globale des sciences.

Ce document ne se substitue pas aux cours d'informatique. Il propose une approche centrée sur les usages en chimie : comprendre un algorithme, choisir une méthode adaptée à une situation donnée et interpréter correctement les résultats obtenus. Les exemples sont volontairement proches des situations rencontrées en cinétique, thermodynamique ou exploitation de données expérimentales.

Ce polycopié est un outil de travail : à utiliser en amont pour préparer une activité ou en aval pour consolider une méthode. L'objectif est de développer l'autonomie des étudiants dans l'usage du numérique au service de la modélisation et de l'analyse en chimie.

Table des matières

NUM·Scriptage : Les bonnes pratiques de scriptage **2**
NUM·Bases : Concepts de base du scriptage python **4**
NUM·Arrays : Analyse et traitement de tableaux de données **5**
NUM·Résolution : Résolution numérique d'équations et de systèmes **12**
NUM·Stats : Traitement statistique et simulations aléatoires **17**

Les bonnes pratiques de scriptage

Python est un langage de programmation populaire, apprécié pour sa simplicité et sa lisibilité. Cependant, pour écrire un code Python de qualité, il est essentiel de suivre certaines bonnes pratiques de scriptage. Ce cours d'une page vous guidera à travers les principes fondamentaux à respecter lors de la rédaction de scripts en Python.

Un script doit être auto-documenté

Cela signifie que son code, ses noms d'objets et sa structure doivent être suffisamment clairs pour comprendre ses fonctions sans dépendre exclusivement de commentaires explicatifs.

1. Le bon choix de nom d'objet

▶ 1

Coder. Utilisez des noms de variables et de fonctions qui décrivent clairement leur but. Évitez les noms courts et non explicites.

En pratique.

- Je vous conseille en plus de commencer le nom de l'objet par son type. Ceci permet de garder trace du type d'objet qu'on manipule, et donc du type de calcul dans lequel on peut l'utiliser.
  Par exemple : `int_invites`*,* `list_invites` ***et*** `fct_invites` *sont trois objets distincts dont on comprend intuitivement qu'il s'agit (sans doute) du nombre d'invités (type* `int`*), d'une list d'invités (type* `list`*, sans doute contenant le nom des invités ?) et d'une fonction liée à un problème d'invités.*
- Les noms trop courts (`a`, `b`, `x`) peuvent facilement prêter à confusion. Les éviter, sauf cas très spécifiques :
  - un exercice de mathématique ou d'informatique pur (racines du polynômes $ax^2 + bx + c$, tri d'une `list` `L`, calcul de $n!$…
  - mais utiliser les us :
    - `m`, `n`, `p`, `q` pour des entiers, `i`, `j`, `k` pour des entiers incrémentés (`for i in range...`),
    - `a`, `b`, `c`, `x`, `y`, `z` pour des réels,
    - `L`, `M` pour les `list`.

2. Le bon usage des commentaires

▶ 2

Coder. Dans un script, tout texte qui suit le symbole `#` n'est pas interprété par Spyder. Il permet donc de laisser des **commentaires** dans le script, ce qui permet de laisser trace de ce que fait le script. Il n'y a pas qu'une bonne solution à un problème en informatique, et il n'y a rien de plus horrible qu'essayer d'interpréter des centaines de milliers de lignes de code que quelqu'un d'autre a écrites, ce qui arrive souvent. Laisser des commentaires est donc une bonne pratique de programmation. C'est aussi un objectif du programme. **Tout script doit être commenté**.

En pratique. Évitez les commentaires redondants qui répètent simplement ce que le code fait déjà clairement. Un bon code devrait être auto-documenté dans la mesure du possible grâce à des noms significatifs et des structures logiques.

Exemple.

```
# Calcul de la moyenne des notes en divisant la somme par le nombre de notes
moyenne = sum(notes) / len(notes)
```

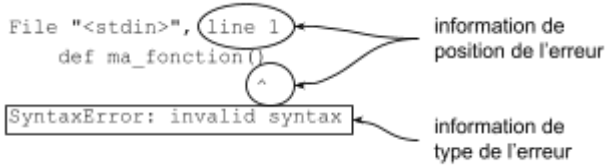
3. Lire les messages d'erreur

Les messages d'erreur en Python sont des outils essentiels pour diagnostiquer et corriger les problèmes dans votre script. Ils fournissent des informations précieuses sur la nature des erreurs et leur emplacement. Cependant, l'interprétation dynamique, telle qu'elle est réalisée Spyder, ajoute une couche supplémentaire d'assistance au processus de scriptage.

Imaginons qu'on script la fonction suivante :

```
def ma_fonction()
    print("Bonjour, monde!")
```

A l'exécution du code, Python renverra un message d'erreur :



3.1. Position de l'erreur

Lorsque vous rencontrez une erreur dans votre code Python, l'un des points de vigilance essentiels est de prêter attention à la position de l'erreur. Python essaie de signaler la ligne et le caractère exacts où l'erreur a été détectée. Cela vous permet de localiser rapidement la source du problème.

Cependant, Python cherche à interpréter le code aussi loin que possible avant de signaler une erreur. Il peut donc être capable d'interpréter le code au-delà de la position réelle de l'erreur, simplement parce que pour lui, la suite a tout de même un sens (mais pas le sens que vous voudriez que ça ait !). Il est donc important de remonter dans le code pour identifier la véritable cause de l'erreur.

Par exemple, l'exécution du script :

```
print("Aller ",3
x=2
```

donne le message d'erreur :

```
  File "<stdin>": line 2
    x=2
    ^
SyntaxError: invalid syntax
```

Or l'erreur n'est pas sur la ligne 2 sur `x`, mais sur la ligne 1 par il manque une fermeture de parenthèse. En fait, Spyder chercher à interpréter la commande `print("Aller ",3x=2`, et la première erreur qu'il voit est l'erreur de syntaxe `3x`, qu'il ne sait pas interpréter.

3.2. Type d'erreurs classiques

1. `SyntaxError` : Ce type d'erreur survient lorsqu'il y a une violation des règles de syntaxe de Python. Cela peut être dû à des parenthèses manquantes, une indentation incorrecte, une utilisation incorrecte des opérateurs, etc.
2. `IndentationError` : Python est sensible à l'indentation, et cette erreur se produit lorsque l'indentation n'est pas correctement alignée. Par exemple, mélanger des espaces et des tabulations dans la même région de code provoquera une erreur d'indentation.
3. `NameError` : Cela se produit lorsque vous essayez d'utiliser une variable ou une fonction qui n'a pas été définie. Assurez-vous que la variable ou la fonction a été correctement initialisée avant de l'utiliser.
4. `TypeError` : Cette erreur survient lorsque vous effectuez des opérations incompatibles sur des types de données, comme l'addition d'une chaîne de caractères et d'un entier.
5. `ValueError` : Cela se produit lorsque le type de données est correct, mais la valeur n'est pas valide. Par exemple, lors de la conversion d'une chaîne en un nombre entier, si la chaîne ne représente pas un nombre valide.
6. `IndexError` : Cette erreur survient lors de l'accès à un index inexistant dans une liste, un tuple ou une séquence.
7. `ZeroDivisionError` : Cela se produit lorsque vous essayez de diviser par zéro, ce qui est mathématiquement invalide.

Cette description sommaire de l'erreur est complétée par des détails qui permettent généralement de cerner le problème rapidement, et de manière autonome. Par exemple, `NameError` est suivi du nom de variable qui n'a pas été définie. Ces messages sont dans un anglais assez transparent. Si cependant vous éprouvez des difficultés à comprendre le message, vous avez accès (en TP) à internet…

Concepts de base du scriptage Python

1. Les briques d'un script

Élément	Rôle	Exemple	Comment reconnaître ?
Valeur	Information manipulée par Python.	`100.2`	Nombre, texte ou ensemble de données écrit directement
Variable	Nom qui permet de stocker une valeur.	`V = 100.2`	Mot
Fonction	Réalise une action ou un calcul.	`mean(V)`	Mot suivi de parenthèses : `nom(...)`
Bibliothèque	Ensemble de fonctions supplémentaires.	`numpy`	Importée au début du script avec `import`
Commentaire	Texte explicatif ignoré par Python.	`# Calcul de la moyenne`	Commence par `#`

2. Les principaux types de données

Type	Exemple	Utilisation
`int`	`3`	nombre entier
`float`	`3.14`	nombre réel
`str`	`"eau"`	texte
`array`	`[99.8, 100.2, 100.1]`	série de mesures

3. Les fonctions

Une fonction se reconnaît par une syntaxe comme nom(argument).

Exemples : `print(V)`, `len(V)`, `numpy.mean(V)`.

Une fonction renvoie souvent une valeur, qui doit donc être stockée pour être utilisée.

```
m = numpy.mean(V)
```

⇒ `numpy.mean(V)` est un float qui vaut la moyenne de `V`, qui est stockée dans la variable `m` (qui est donc un float).

4. Les règles à connaître

- Python lit le script de haut en bas.
- Une variable ou une fonction doit être définie avant d'être utilisée.
- `=` signifie « on stocke une valeur dans une variable », et non « est égal à ».
- `m = numpy.mean(V)` : la valeur renvoyée par `numpy.mean(V)` est stockée dans `m`.
- La syntaxe doit être exacte (parenthèses, virgules, deux-points, majuscules...).

5. Lire un script : par où commencer ?

Avant de modifier un programme, repérer :

1. les bibliothèques importées ;
2. les fonctions utilisées ;
3. les variables contenant les données expérimentales ;
4. les calculs réalisés ;
5. les résultats affichés.

En cas d'erreur...

Vérifier en priorité :

- les parenthèses ;
- les virgules ;
- les deux-points (:) ;
- l'indentation ;
- l'orthographe des noms de variables.

Analyse et traitement de tableaux de données

I. Le type array

✱ 14

Définition. Une variable de type **array** sert à stocker des séquences ordonnées de valeurs. Ce type n'est pas natif en python, il faut préalablement l'importer via la bibliothèque `numpy`.

En pratique. En IPT, vous travaillez avec les `list`. En sciences, on préfère travailler avec des `arrays`. Ces deux types se ressemblent de prime abord, mais certaines caractéristiques des `arrays` sont très appréciées lorsqu'il s'agit de traiter des données numériques.

▶ 15

Coder. On forme un `array` en convertissant une `list` avec la fonction `np.array()`. Chaque élément de l'`array` est indexé, le premier indice est 0, puis 1 etc.

Les syntaxes qui sont identiques ou similaires à celles sur les `list`

Syntaxe	Type de valeur renvoyée	Exemple	Traduction
`import numpy as np`	—		importe la bibliothèque `numpy` et renomme-la `np`.
`np.array([` *valeur* **`,`** *valeur* **`,`** *valeur* **`])`**	*array*	`ar_t = np.array([0,1,2,3,4,5])`	l'**`array`…**
`len(` *array* **`)`**	*int*	`len(ar_t)` renvoie `6`	le **nombre d'éléments de l'`array`**
array **`[`** *int* **`]`**	selon le contenu de l'*array*	`ar_t[3]` renvoie `2`	l'**élément d'indice** … **de l'`array`**
array **`[`** *int* **`:`** *int* **`]`**	*array*	`ar_t`**`[0:3]`** renvoie l'`array [0,1,2]`	la **tranche de l'`array` entre les indices** … **inclus et** … **exclus**
array **`[`** *int* **`] =`** *valeur*	aucune	`ar_t[1]=1.5` ne renvoie rien, mais après son exécution, `ar_t` vaut `[0,1.5,2,3,4,5]`.	**modifie l'élément d'indice** … **de l'`array`**

Syntaxe	Exemple	Commentaire
`for` *variable* **`in`** *array* **`:`** *[commande 1]* *[commande 2]*	`for element in ar_t :` `  print(element)`	**Une boucle `for` itère sur n'importe quelle valeur structurée** : par exemple les types `range`, mais aussi les `array`. A la première itération, `element` vaut `0`, à la seconde, il vaut `1`, etc.

▶ **Coder.**

15

Les syntaxes qui sont différentes de celles sur les `list` (et c'est tout l'intérêt des `arrays`)

Syntaxe	Type de valeur renvoyée	Exemple	Traduction

fonction **(** array **)**	*array*	`abs( ar_t )`
		`1 / ar_t`
		`np.exp(ar_t)`
			applique la fonction élément par élément

`np.zeros(` *int* **`)`**	*array*	`np.zeros(5)` renvoie `[0,0,0,0,0]`	**arrays de ... zéros**
`np.linspace (` *float* **`,`** *float* **`,`** *int* **`)`**	*array*	`np.linspace(0,100,1000)` renvoie `[0, 0.1, 0.2, 0.3,...,99.8, 99.9]`	**`array de int valeurs régulièrement espacés allant de ... inclus à … exclus`**

array **+** *array*
array **–** *array*
array ***** *array*
array **/** *array*
int ***** *array*
	array	`np.array([0,1,2])+np.array([10,20,30])`
		renvoie `[10,21,32]`
		
		`3*np.array([0,1,2])` renvoie `[0,3,6]`
			fais l'opération élément par élément

`np.mean(` *array* **`)`**	*float*	`np.mean(ar_t)`	**moyenne** des éléments
`np.std(` *array* **`)`**	*float*	`np.std(ar_t)`	**écart-type** des éléments

Vigilance. Python indice les éléments en comptant à partir de 0. Ainsi, l'élément d'indice 3 est le 4ème élément de la `list`.

2. Tracé de fonctions

▶ **Coder.**

1

Syntaxe	Exemple	Commentaire
revoir les syntaxe d'import de module, si besoin	`import matplotlib.pyplot as plt`	Ici, on voit bien l'intérêt d'introduire l'alias `plt`, bien plus court que `matplotlib.pyplot`.
`plt.plot(` array , array `)`	`plt.plot( X ,Y )`	Affiche un nuage de point, la première `list` portant les abscisses, la seconde les ordonnées.
`plt.plot(` array , array , str `)`	`plt.plot( X , Y ,"o--r")`	On peut préciser le style de représentation des données comme `str` dans l'ordre marqueur–ligne–couleur : ● marqueurs : `.` point, `o` cercle, `^` triangle, `s` carré, `x` croix, `+` plus ● lignes : `-` continu, `--` tirets, `:` pointillés ● couleurs : `b` bleu, `r` rouge, `g` vert, `k` noir

`plt.xlabel(` str `)`
`plt.ylabel(` str `)`
`plt.title(` str `)`
	`plt.xlabel("abscisses")`
	`plt.ylabel("ordonnées")`
	`plt.title("Titre du graphique")`
		Ces fonctions permettent de définir le titres des axes et du graphique.

`plt.axis(` array de 4 float `)`	`plt.axis([xmin,xmax,ymin,ymax])`	Permet de définir les limites des axes des abscisses et des ordonnées.
`plt.bar(` array , array `)`	`plt.bar( X ,Y )`	Trace un histogramme dont le premier argument est la liste des abscisses, et le second la liste des hauteurs des barres.
`plt.show()`	`plt.show()`	Affiche la représentation (pas forcément nécessaire, ça dépend de l'OS et de la version de Spyder).

3. Dérivation numérique

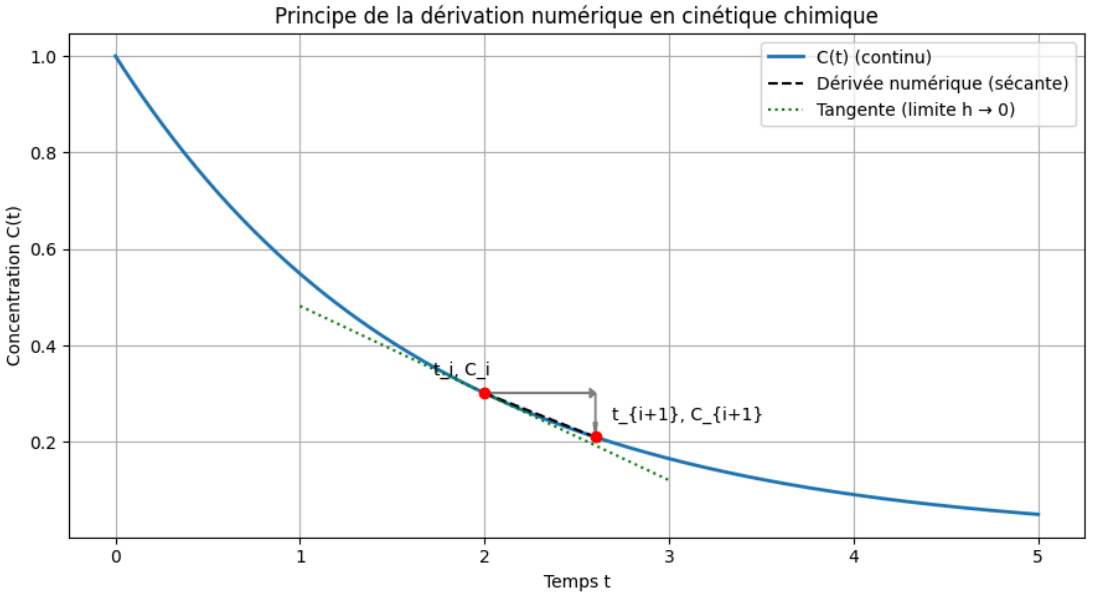
✱ 6

Propriétés. Différence entre dérivée d'une fonction mathématique et dérivée de données numériques.

En maths, on a accès à un nombre infini de données (fonction continue). De là, on peut définir la dérivée de la fonction f en l'abscisse x par l'expression : $f'(x) = \lim_{h \to 0^+} \frac{f(x+h)-f(x)}{h}$. On voit qu'au numérateur on a une différence d'ordonnées (entre deux points très proches), et au dénominateur la différence d'abscisses correspondante (car $(x + h) - (x) = h$). Il s'agit donc de la pente de la tangente à la courbe en l'abscisse x.

En sciences naturelles, on n'a accès qu'à un nombre fini (on dit "discret") de données. On ne peut donc pas faire tendre h vers 0, mais on peut quand même le rendre le plus petit possible (en cohérence avec les données à disposition).

Considérons qu'on a N points expérimentaux, indicés par des entiers i allant de 0 à N–1 ; par exemple, des valeurs de concentration C_i à différents instants t_i. On peut donc estimer la dérivée temporelle de C à l'instant t_i en calquant l'expression mathématique ci-dessus : $\frac{dC}{dt}(t_i) \approx \frac{C_{i+1}-C_i}{t_{i+1}-t_i}$: on retrouve bien une petite variation de concentration au numérateur, et une petite variable d'abscisses au dénominateur. Les grandeurs indicées i et i+1 correspondent à deux points successifs.



▶ 6

Coder. Calculer une dérivée numérique en python.

Soit `C` un array de `N` valeurs de concentration (indicées de `0` à `N-1`), et `t` l'array de dates correspondantes (mêmes indices). Les valeurs de dérivées numériques peut être obtenue sous la forme d'un array `derC`, de taille `N`, en utilisant la fonction `gradient` de la bibliothèque `numpy`, qu'il convient donc d'importer préalablement.

```
import numpy as np

t =  array([0, 10, 20, 30, 40, 50, 60])
C =  array([1, .9, .8, .7, .6, .5, .4])

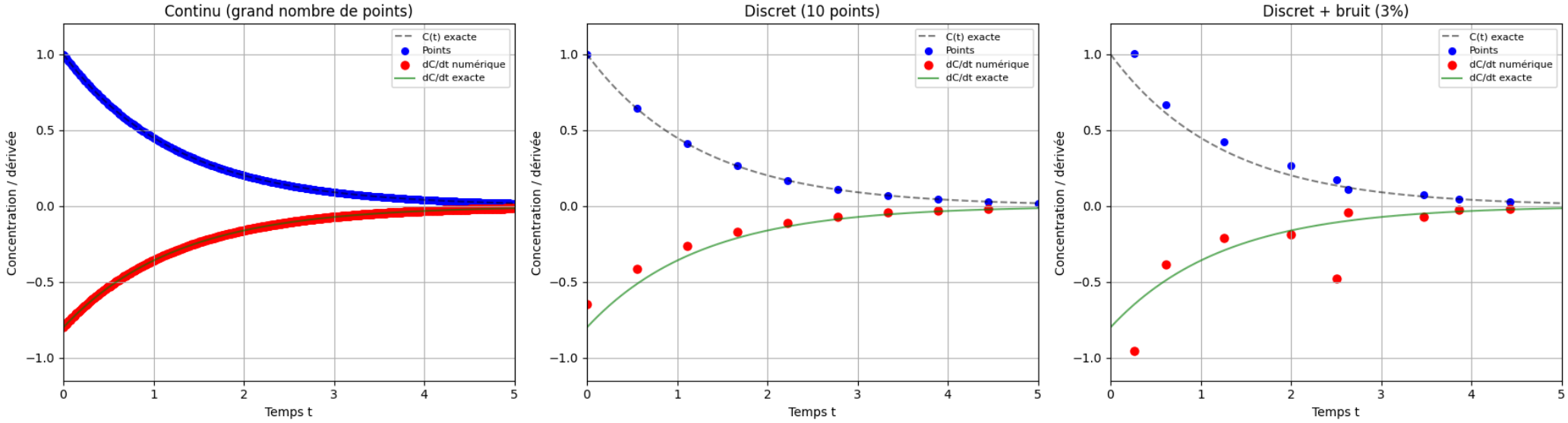
dC_dt = np.gradient(C,t)          # calcul de la dérivée dC/dt, et stockage dans dC_dt
```

En pratique. Si on s'en tient à l'encadré précédent, on devrait s'attendre à n'avoir que N–1 valeur de dérivée, car la dérivée est définie comme une différence entre deux points successifs : on va avoir une valeur de dérivée de moins que de points expérimentaux. Soyons honnête, ça serait embêtant car il faudrait alors modifier la liste des abscisses pour qu'elle ait la même taille en vue d'en faire un graphe (sinon python plante).
En fait, la fonction gradient est un peu plus subtile. Sans entrer dans les détails, elle donne bien un array de taille N. Et c'est bien pratique !

Précision : bruitage

La dérivée mathématique s'appuie sur un **nombre infini de valeurs** **définies de manière précise** : ces deux points ne sont pas possibles en sciences. Par conséquent :

- le calcul d'une dérivée donne des résultats pertinents à condition d'**avoir un nombre important de points expérimentaux**, suffisamment rapproché sur l'axe des abscisses.
- les fluctuations des mesures impliquent d'assez fortes fluctuations sur les dérivées. **Les dérivées ont l'air très bruitées**.



Un exemple issu de la cinétique

```
import numpy as np                # nécessaire pour utiliser les arrays

t = np.array([0,10,20,30,40])                  # points expérimentaux temps (s)
C = np.array([1.00,0.82,0.67,0.55,0.45])       # points expérimentaux concentration (mol/L)
N = len(t)                        # nombre de points, correspond à la taille (length) de t (ou de C)

derC = np.array([(C[i+1]-C[i])/(t[i+1]-t[i]) for i in range(N-1)])
                                  # approx dC/dt par différences finies, implémentée par compréhension
v = -derC                         # vitesse de réaction
```

4. Diagrammes de distribution

On développe ici cette section sur la base d'un diagramme de distribution pour le couple $AcOH/AcO^-$ ($pK_a = 4,8$). La dernière partie explicite les différences d'avec les diagrammes de distribution en cas de précipitation (exemple AgCl).

1. Stratégie de mise en équation

- On fixe une valeur de pH, donc $[H_3O^+]$ est connu
- On exprime les fractions de distribution (ici notées f_{Acide} et f_{Base}, en utilisant la formule de Henderson–Hasselbalch ($pH = pK_a + \log\left(\frac{f_{Base}}{f_{Acide}}\right)$ et la conservation de la matière ($f_{Acide} + f_{Base} = 1$). Ainsi, pour un monoacide $AcOH/AcO^-$, on trouve :

$$f_{Acide} = \frac{1}{1+10^{pH-pK_a}} \quad \text{et} \quad f_{Base} = 1 - f_{Acide}$$

- On balaie un array de valeur de pH : ça peut être fait avec une boucle `for` explicite, ou utiliser les propriétés des arrays si pH est stockée sous forme d'arrays.

Remarque : pour un diacide, même logique avec plusieurs pK_a (pas détaillé ici).

2. Script Python (acide-base)

```
import numpy as np              # nécessaire pour utiliser les arrays

pKa = 4.8                       # pKa du couple d'intérêt
pH = np.linspace(0,14,200)      # arrays d'abscisses (pH) : 200 points entre pH=0 et pH=14

fAcide = 1/(1 + 10**(pH-pKa))   # fraction d'acide. pH est un array, pKa un int
fBase = 1 - fAcide              # fraction AcO-

# Ajouter le tracé ici (pH en abscisse, fractions en ordonnée)
```

3. Spécificités en cas de précipitations

Différences clés :

- On ne raisonne plus avec un pK_a, mais avec un pK_s
- L'axe des abscisses n'est plus pH (ici, on traitera avec pCl)
- Le précipité n'existe pas toujours :
  - Si (`if`) la condition de précipitation $\left(\frac{[Ag^+]_0 \cdot [Cl^-]_0}{(C°)^2} > K_s\right)$ est vérifiée, l'équilibre est atteignable $\left(\frac{[Ag^+] \cdot [Cl^-]}{(C°)^2} = K_s\right)$, donc :
    $$f_{Ag^+} = \frac{[Ag^+]}{C_{tot}} = \frac{(C°)^2 \cdot 10^{pCl - pK_s}}{C_{tot}}$$
  - Sinon (`else`), tout est sous forme Ag^+ : $f_{Ag^+} = 1$.
- Pour chaque valeur de fraction de Ag^+ f_{Ag^+}, la fraction de solide f_{AgCl} est $f_{AgCl} = 1 - f_{Ag^+}$.

Stratégie :

- On fixe une valeur de pCl, donc $[Cl^-]$ est connu
- On teste la condition de précipitation, et calcule f_{Ag^+} selon la bonne formule selon le cas
- On balaie un array de valeur de pCl avec une boucle `for` explicite (dans ce cas, comme il y a disjonction de cas, on ne peut pas utiliser les propriétés des arrays facilement).

```
import numpy as np         # nécessaire pour utiliser les arrays

pKs = 9.7                  # pKs du solide AgCl
Ag_0 = 1e-2                # Concentration de tracé (et conc° totale) en Ag+ (e = 10-puissance)
pCl = np.linspace(0,6,200) # arrays d'abscisses (pCl) : 200 points entre pCl=0 et pCl=6

fAg = np.array([])         # initialise un array fAg pour stockage par append successifs.

for pCl_valeur in pCl :    # balaie toutes les valeurs d'abscisses, appelées pCl_valeur
       if ( Ag_0 )* ( 10**(-pCl_valeur) ) > ( 10**(-pKs) ) :# teste s'il peut y avoir précipitation
              fAg_valeur = (10**(pCl-pKs))/Ag_0             # Si oui, calcule + stocke sous fAg_valeur
       else :
              fAg_valeur = 1                                # Si non, calcule + stocke sous fAg_valeur
       fAg.append( fAg_valeur )          # fAg_valeur est la fraction d'Ag+ pour la valeur pCl_valeur
                                         # qu'on stocke par méthode append dans l'array fAg
# A la fin de la boucle for, fAg contient toutes les valeurs d'ordonnées (fraction d'Ag+)
fAgCl = 1-fAg              # Calcule la fraction d'AgCl correspondante (♥♥♥ arrays ♥♥♥)

# Ajouter le tracé ici (pCl en abscisse, fractions en ordonnée)
```

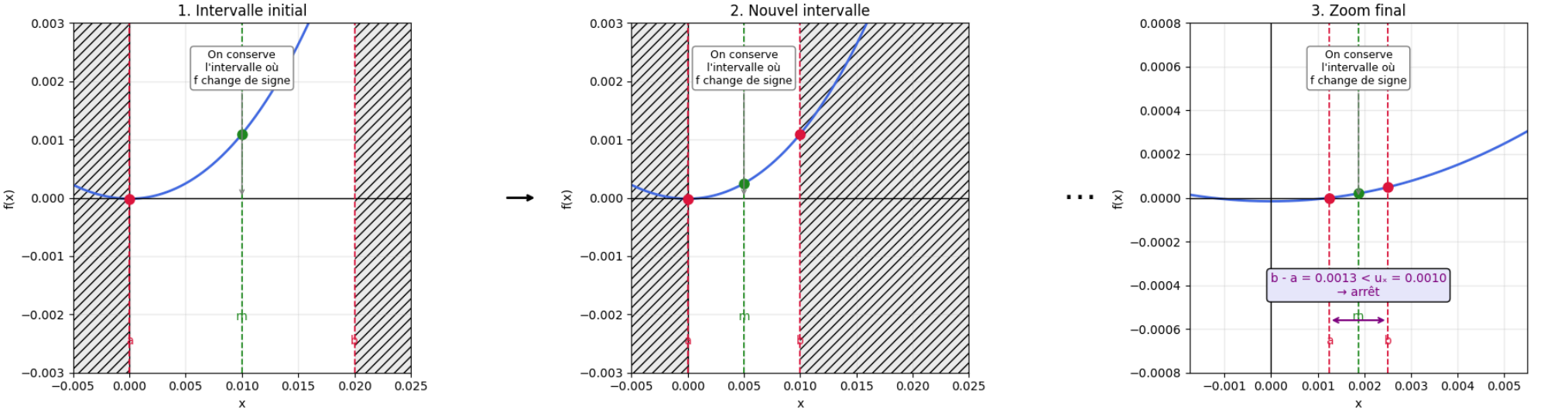
RÉSOLUTION NUMÉRIQUE D'ÉQUATIONS ET DE SYSTÈMES

ÉQUATIONS NON LINÉAIRES À UNE VARIABLE. MÉTHODES DE RECHERCHE DE RACINES

I. STRATÉGIE DE MISE EN ÉQUATION

Traitons l'exemple de la recherche de l'avancement à l'équilibre de la dissolution de AcOH dans l'eau ($C_0 = 0,1$ mol·L^{-1}, $K° = 10^{-4,8}$), d'équation $AcOH + H_2O = AcO^- + H_3O^+$.

- Avec python, on peut résoudre une équation du type $f(x) = 0$ (deux méthodes au programme, *cf. infra*). Il peut donc nécessiter d'avoir un peu de réécriture. Par exemple, l'équilibre chimique ($Q_r(x) = K°$) doit donc être réécrit $Q_r(x) - K° = 0$. Ainsi, la fonction f dont on cherche la racine est $\left(Q_r(x) - K°\right)$. A l'aide d'un tableau d'avancement et de l'expression de Q_r, on peut écrire qu'on cherche la racine de $f(x) = \frac{x^2}{C°(C_0 - x)} - K°$.
- Le principe de la dichotomie repose sur la propriété qu'une fonction continue change de signe quand elle s'annule.
- Plus précisément :
  - On part du principe qu'on a une fonction f continue sur un intervalle [a,b]
  - Si $f(a)$ et $f(b)$ sont de signes opposés (ce qu'on traduit par la condition $f(a) \times f(b) < 0$), alors il existe au moins une racine dans l'intervalle [a,b].
  - On tente notre chance au milieu ($m = \frac{a+b}{2}$) :
    - Si par hasard, on a trouvé la racine ($f(m) = 0$), on a fini le travail : m est la racine cherchée
    - Sinon, on conserve le sous-intervalle ([a,m] ou [m,b]) où le changement de signe persiste
  - On recommence sur l'intervalle plus petit autant de fois qu'il faut.
- On a donc besoin d'itérer ce calcul un nombre de fois inconnu au départ : boucle `while` (boucle conditionnelle)



2. Méthode par dichotomie explicite

L'idée est de mettre en place l'algorithme ci-dessus. Dans l'exemple donné ci-dessous, je propose un script très simplifié, pour lequel je pars du principe qu'il y a une racine dans l'intervalle de recherche… on peut peaufiner en mettant des vérifications initiales.

```
K = 10**(-4.8)             # constante d'équilibre
C0 = 0.1                   # concentration initiale
u_x = 0.001                # précision à laquelle on veut connaître la valeur de la racine... ça sera
                           # nécessaire pour savoir quand arrêter la recherche de la racine !
def f(x) :                 # définition de la fonction dont on cherche la racine (f(x) = Qr(x) - K)
       return x**2/(C0-x) - K

a,b = 0, C0  # initialisation de l'intervalle de recherche de la racine (x doit être entre 0 et C0)

while abs(a-b)>u_x :       # itère tant que la précision n'est pas atteinte
       m = (a+b)/2         # calcule l'abscisse du milieu de l'intervalle de recherche
       if f(a)*f(m) <= 0:  # si la racine est dans [a,m]...
              b = m        # remplace l'intervalle de recherche [a,b] par [a,m]
       else:               # sinon ...
              a = m        # remplace l'intervalle de recherche [a,b] par [m,b]
# Le script boucle sur le while en réduisant l'intervalle de recherche de moitié à chaque itération
# Une fois la boucle terminée, on a un tout petit intervalle [a,b] qui contient la racine

x = (a+b)/2                # on estime la solution comme milieu de [a,b] (on aurait pu prendre a ou b)
print("x =",x)             # affichage de la racine, qui est ici l'avancement à l'équilibre
```

3. Méthode avec bisect du module scipy.optimize

La fonction bisect du modul scipy.optimize procède à la recherche dichotomique "automatiquement"... mais faites-vous confiance à une boîte noire ?

```
from scipy.optimize import bisect       # nécessaire pour utiliser bisect

K = 10**(-4.8)             # constante d'acidité
C0 = 0.1                   # concentration initiale

def f(x) :                 # définition de la fonction dont on cherche la racine (f(x) = Qr(x) - K)
       return x**2/(C0-x) - K

x = bisect(f,0,C0)         # recherche de racine de f dans l'intervalle [0,C0]
print("x =",x)             # affichage de la racine, qui est ici l'avancement à l'équilibre
```

Système d'équations différentielles non linéaires. Méthode d'Euler

I. Principe de la méthode d'Euler

✱ 19

Définition. La **méthode d'Euler** est une méthode d'estimation numérique de solution de système d'équations différentielles, qui repose sur (pour des équations temporelles) :

- la **discrétisation du temps** : la grandeur infiniment petite dt est considérée seulement très petite : c'est le **pas de discrétisation**. Ainsi, au lieu d'avoir une infinité de point pour $f(t)$, on aura que des points discrets espacés de dt. (*N.B.* : "discret" est l'opposé de "continu" : ici, la fonction continue $f(t)$ est remplacée par son estimation discrète).
- l'**estimation par itérations successives** de la solution. Ainsi $f(t + dt)$ est estimée à partir de la valeur de $f(t)$ en utilisant un développement de Taylor d'ordre 1 : $f(t + dt) \approx f(t) + \frac{df}{dt}(t) \times dt$
  Des conditions initiales sont donc nécessaires.

Une illustration est donnée à la page suivante.

En pratique.

- Les instants discrets et les valeurs discrètes des fonctions sont stockés dans des `list` ou `arrays`.
- Les calculs reposent sur l'approximation suivante, si on note `i` les indices successifs de ces `list`, et `der_f(...)` une fonction qui calcul la dérivée de f (ses arguments dépendent fortement du cas étudié) :

  `f[i+1] = f[i] + der_f(...) * dt`
- Vu que les fonctions ne sont, en pratique, pas utilisée ailleurs dans le script, on peut aussi éviter la définition de fonction et en faire les calculs directement dans la boucle for :

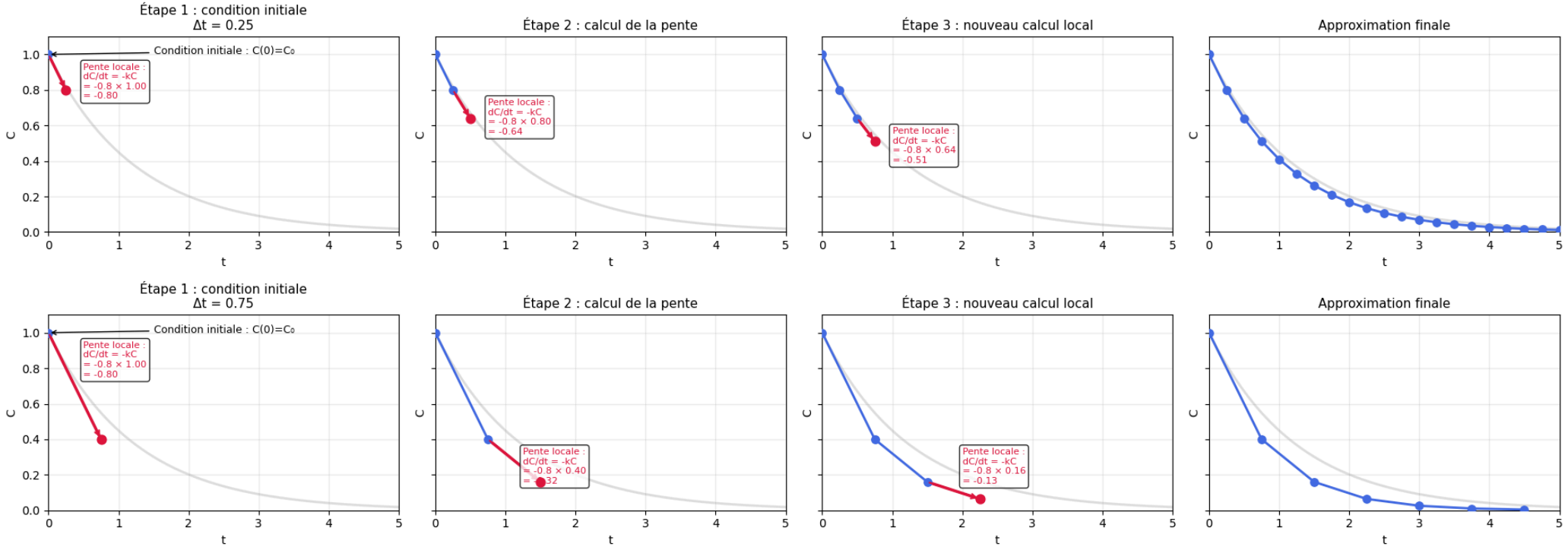
```
for i in range(N) :
   dC_dt = -k*C[i]                # Calcule la pente locale au i-ième point
   C.append( C[i] + dC_dt * dt )  # Implémente méthode d'Euler, stocke dans list C
```

☑ 20

Méthode. Implémentation de la méthode d'Euler en Python, sur un exemple de chimie.

1. Établir le système d'équations différentielles sur les bases de vos connaissances de chimie, en utilisant les outils mathématiques.
2. Concevoir les fonctions Python qui renvoient la valeur de ces dérivées, ***e.g.*** `der_C1(C1,C2)`, `der_C2(C1,C2)`.
3. Concevoir le script de résolution par méthode d'Euler, dont les éléments principaux sont :
   a. l'établissement des conditions initiales ;
   b. une boucle itérative (typiquement, `for`) dans laquelle on va retrouver pour chaque fonction de l'équation différentielle : `f1[i+1] = f1[i] + der_f1(f1[i],f2[i]) * dt` etc…

Méthode d'Euler appliquée à une cinétique d'ordre 1
Équation différentielle : dC/dt = -kC ; condition initiale : C(0)=C₀



2. Exemple d'implémentation en cinétique

```
k1 = 2.0      # Définition des constantes de vitesse
k2 = 0.5
A0 = 1.0      # Définition des concentrations initiales
B0 = 0.05
C0 = 0.0

dt = 0.01     # Pas de temps, en s : plus il est petit, plus Euler est précis
t_max = 10    # Temps final de la simulation, en s
N = int(t_max / dt)   # Détermination du nombre de points de calcul

t = [0]   # Initialisation des list de stockage avec les conditions initiales
A = [A0]
B = [B0]
C = [C0]

# ==============================
# MÉTHODE D'EULER
# ==============================

for i in range(N):
    # 1. Calcul des pentes locales (dérivées). On traduit ici en langage mathématique le mécanisme réactionnel
    dA_dt = -k1 * A[i] * B[i]
    dB_dt = k1 * A[i] * B[i] - k2 * B[i]
    dC_dt = k2 * B[i]

    # 2. Mise à jour par Euler : valeur suivante = valeur actuelle + pente × pas de temps
    A.append( A[i] + dA_dt * dt )
    B.append( B[i] + dB_dt * dt )
    C.append( C[i] + dC_dt * dt )

# Ensuite, on peut passer à l'affichage...
```

TRAITEMENT STATISTIQUE ET SIMULATIONS ALÉATOIRES

I. RÉGRESSION LINÉAIRE

* 33

Coder.

- Une régression peut être implémentée par la **fonction `polyfit()` du module `numpy`**.
- La fonction `polyfit(X,Y,deg)` prend 3 arguments :
  - `X` : une list ou un array d'abscisses
  - `Y` : une list ou un array d'ordonnées (de même taille que `X`)
  - `deg` : un entier qui définit le degré du polynôme qu'on cherche à ajuster. Comme on ne fait que des régressions affines, on utilise toujours `1`.
- La fonction `polyfit()` renvoie les paramètres de régression, par ordre décroissant de puissance. En pratique, pour un régression linéaire ($y = ax + b$), elle renvoie dans cette ordre a,b (et donc pente puis ordonnée à l'origine).

En pratique.

- Stocker les paramètres de régression pour utilisation : **`a,b = np.polyfit(X,Y,1)  # a pente, b ord. à origine`**
- En général, on cherche aussi à tracer un graphique portant les points expérimentaux et la courbe de régression. La courbe de régression est simplement l'image par la fonction affine des abscisses expérimentales. Par exemple, si X est un array, on peut le faire directement : `plt.plot(X,a*X+b)`

Exemple. Voici ci-dessous un exemple de script qui permet d'effectuer une régression linéaire, d'afficher le graphique et d'afficher les paramètres de régression.

```
1.    import numpy as np                                    # importe numpy pour polyfit()
2.    import matplotlib.pyplot as plt                       # importe plt pour plot()
3.
4.    # Données expérimentales (exemple concentration et absorbance), en array (plus facile)
5.    C = np.array([0.0, 0.1, 0.2, 0.3, 0.4, 0.5])          # en mol/L
6.    A = np.array([0.00, 0.12, 0.23, 0.31, 0.41, 0.52])    # sans unité
7.
8.    # On effectue la régression linéaire A = a*C + b
9.    a,b = np.polyfit(C, A, 1)                             # a = pente, b = ord. à l'origine
10.
11.   # Affichage des paramètres de régression
12.   print(a,b)                                            # on peut faire plus joli...
13.
14.   # Affichage d'un graph, utilise pour vérifier PARAM
15.   plt.plot(C, A, 'o', label='Exp.', color='blue')       # définit o bleu pour mesures exp.
16.   plt.plot(C, a*C+b, '-', label='RégLin', color='red')  # calc.+déf. trait rouge pour Réglin
17.   plt.title("Absorbance vs. concentration")             # définit titres et axes
18.   plt.xlabel("Concentration (mol/L)")
19.   plt.ylabel("Absorbance (A)")
20.   plt.legend()                                          # affiche la légende du graph
21.   plt.show()                                            # affiche du graph
```

Vigilance. "Ajuster" se dit *fit* en anglais, d'où *polyfit* : on *ajuste* une fonction *poly*nômiale aux données expérimentales.

2. Moyenne et écart-type

✱ 33

Coder.

Syntaxe	Type de valeur renvoyée	Exemple	Traduction
`np.mean(` *array* `)`	*float*	`np.mean(ar_t)`	**moyenne** des éléments
`np.std(` *array* `)`	*float*	`np.std(ar_t)`	**écart-type** des éléments

En pratique.

- En TP, il est relativement rare d'avoir à calculer une moyenne ou un écart-type pour une grandeur isolée (une concentration, une constante d'équilibre, etc.), et encore plus rare de devoir recourir à Python uniquement pour ce type de calcul.
- En revanche, ces outils prennent tout leur intérêt dans le cadre de méthodes numériques comme la méthode de Monte-Carlo, utilisée pour estimer des incertitudes. Cette approche repose sur la simulation d'un grand nombre de réalisations expérimentales. La moyenne des résultats permet d'estimer la valeur centrale, tandis que l'écart-type fournit une mesure de la dispersion des valeurs (on assimile alors, dans ce contexte, écart-type et incertitude-type).
- Il existe plusieurs façons de réaliser ces calculs en Python. Nous avons vu qu'il était possible de travailler avec des tableaux(`arrays`) et d'utiliser les fonctions `mean()` et `stdev()` du module `numpy`. Il existe également les méthodes `.mean()` et `.std()` pour les tableaux (si on préfère une programmation orientée objet).

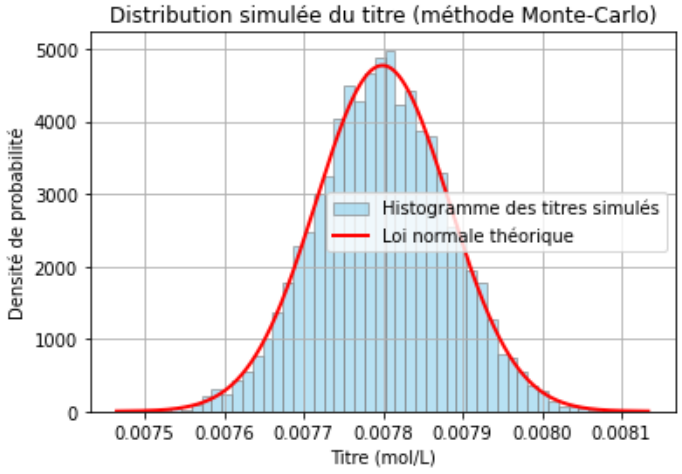
3. Méthode Monte-Carlo

✱ 33

Définition. La **méthode de Monte-Carlo** est une technique d'estimation d'incertitude utile quand les techniques analytiques de propagation d'incertitudes sont trop complexes ou inapplicables : grand nombre de variables et/ou opérations autres que produits et somme (*e.g.*, logarithme, racine carrée). Cette méthode est basée sur une simulation aléatoire d'un grand nombre d'expériences similaires à celle effectivement réalisée au laboratoire.

Propriétés.

- Quand vous pesez un échantillon, vous obtenez une masse qui n'est pas la **valeur vraie** de la masse de l'échantillon. Si la vrai masse est de 100 g, il y a des chances que vous mesuriez 99 g, ou 102 g, dû aux incertitudes de mesure, mais *a priori* il y a plus de chance qu'on soit proche de la valeur vraie que loin.
- Basé sur les lois de probabilité, on peut démontrer qu'une mesure suit une **loi normale** (gaussienne) centrée sur la valeur mesurée, avec un écart-type égal à l'incertitude-type : plus l'incertitude est grande, plus la gaussienne est "épaisse".
- L'idée de la méthode Monte–Carlo est de simuler un grand nombre d'expériences, pour obtenir un grand nombre d'estimations de la valeur vraie. La valeur retenue est alors la moyenne de ces estimations, et l'incertitude-type associée est l'écart-type de ces estimations.



En pratique. On appelle ici "grandeur de sortie" la grandeur dont on cherche à estimer l'incertitude et "grandeurs d'entrée" les grandeurs dont on sait estimer l'incertitude (indication constructeur, erreur de lecture…).
Les étapes d'implémentation en Python sont les suivantes :

1. Exprimer la grandeur de sortie en fonction des grandeurs d'entrée. *Exemple : Pour un titrage :* $C_{titré} = \frac{C_{titrant} \cdot V_{ft}}{V_{prise}}$.
2. Générer aléatoirement, à l'aide de la loi normale, un grand nombre de valeurs pour chacune des grandeurs d'entrée.
3. Simuler un grand nombre d'expériences à l'aide de ces valeurs générées, afin d'obtenir un grand nombre de valeurs pour la grandeur de sortie.
4. Calculer l'écart-type de ces valeurs, qu'on confond avec l'incertitude-type de la grandeur.

✱
33

Exemple. Voici ci-dessous un exemple de script qui permet d'évaluer, par la méthode Monte–Carlo, l'incertitude-type pour un titrage.

```
1.    import numpy as np
2.
3.    ## Fonction pour calculer la grandeur de sortie dont on veut l'incertitude
4.    def calcul_titre(C_titrant, V_prise, V_ft):
5.        return C_titrant * V_ft / V_prise
6.
7.    ## Paramètres expérimentaux
8.    C_titrant = 0.01          # Concentration du titrant (en mol/L)
9.    V_prise = 20.0            # Volume de prise (en mL)
10.   V_ft = 15.60              # Volume de fin de titrage (en mL)
11.
12.   ## Génération d'échantillons aléatoires pour les paramètres expérimentaux
13.   nb_samples = 10000        # Nombre de lancements
14.   C_titrant_samples = np.random.normal(C_titrant, 0.0001, nb_samples)
15.   V_prise_samples = np.random.normal(V_prise, 0.06, nb_samples)
16.   V_ft_samples = np.random.normal(V_ft, 0.03, nb_samples)
17.         # np.random.normal(valeur_centrale,incertitude,nb_de_lancements)
18.         # les variables *_samples sont des arrays, collection d'estimations
19.
20.   ## Calcul des teneurs pour chaque échantillon
21.   titre_samples = calcul_titre(C_titrant_samples, V_prise_samples, V_ft_samples)
22.
23.   ## Calcul et affichage de l'incertitude-type
24.   incertitude = np.std(titres_samples)
25.   print("Incertitude sur le titre :", incertitude)
```

Vigilance.

- La fonction `random.normal()` nécessite que le module `numpy` soit importé préalablement.
- Cette fonction génère un grand nombre de valeurs sous la forme d'un **`array`**.

4. Régression linéaire et Monte-Carlo

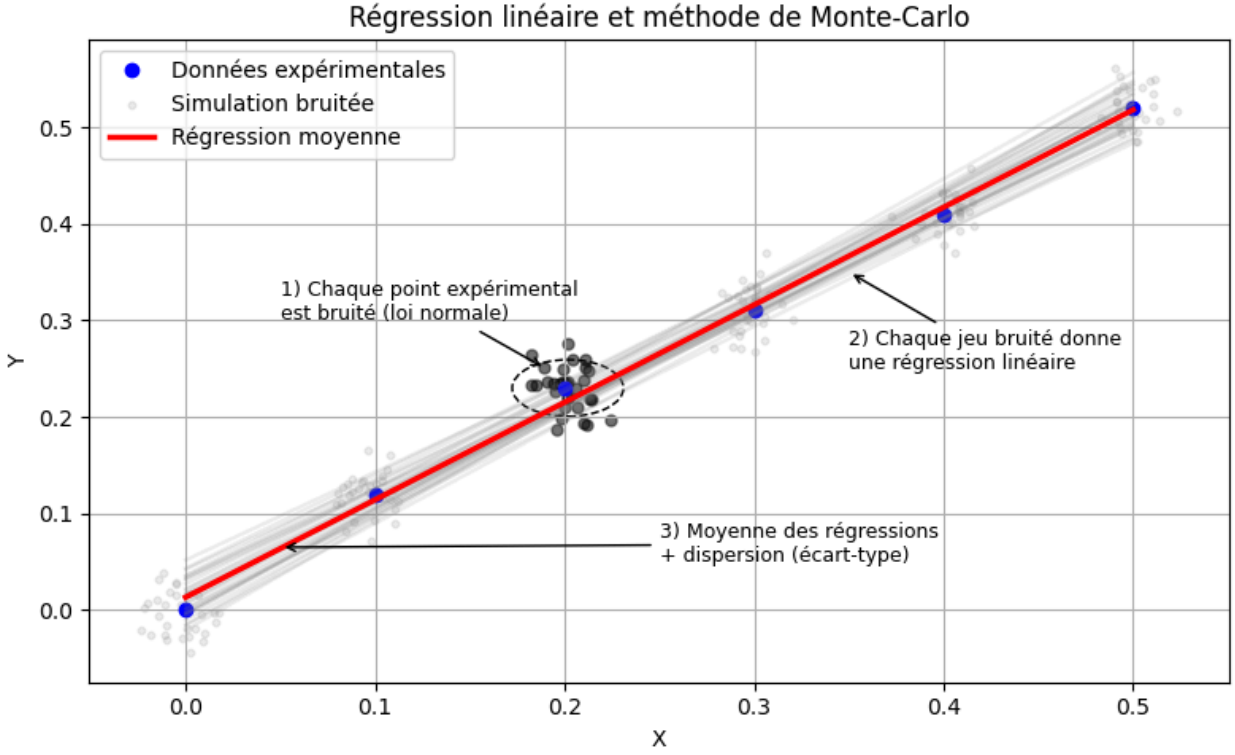
✱ 33

Définition. Dans de nombreux cas, on souhaite utiliser un ou plusieurs paramètres de régression linéaire : par exemple la pente en méthode intégrale pour un ordre 1 correspond à l'opposé de la constante de vitesse. Seulement, comme les données expérimentales viennent avec des incertitudes, celles-ci doivent influer sur la précision de la pente. Coupler la méthode Monte-Carlo avec la régression linéaire permet d'obtenir des incertitudes-type sur les paramètres de régression.

En pratique. On considère un jeu de données expérimentales reliant deux grandeurs X et Y, que l'on suppose liées par une relation affine : Y=aX+b. Les points expérimentaux X et Y présentent des dispersions liées aux incertitudes de mesure. L'idée est alors de :

1. Simuler un grand nombre de jeux de données compatibles avec ces incertitudes (par exemple, avec la fonction `random.normal()` du module `numpy`) ;
2. Réaliser une régression linéaire pour chaque jeu simulé (par exemple, avec la fonction `polyfit()` du module `numpy`) ;
3. Calculer moyenne et écart-type de la distribution des paramètres de régression linéaire (par exemple, avec les fonctions `mean()` et `std()` du module `numpy`)

On obtient ainsi une estimation plus robuste de la pente et de l'ordonnée à l'origine, assortie d'une incertitude (via leur écart-type).



✱ 33

Exemple. Voici ci-dessous un exemple de script qui permet d'évaluer, par la méthode Monte–Carlo, l'incertitude-type pour un titrage.

```
import numpy as np
import matplotlib.pyplot as plt

# Données expérimentales initiales
C = np.array([0.0, 0.1, 0.2, 0.3, 0.4, 0.5])
A = np.array([0.00, 0.12, 0.23, 0.31, 0.41, 0.52])
uC = 0.01     # Incertitude pour la concentration
uA = 0.02     # Incertitude pour l'absorbance

nb_samples = 5000   # nombre d'itération Monte-Carlo

a_values = []       # Initialisation des list de stockage des paramètres de régression linéaire
b_values = []

for _ in range(nb_samples):
    C_sim = np.random.normal(C, uC)    # Génération de jeux bruités pour C puis A
    A_sim = np.random.normal(A, uA)

    a, b = np.polyfit(C_sim, A_sim, 1)    # Régression linéaire sur ce jeu bruité

    a_values.append(a)    # Stockage des paramètres de régression linéaire dans les list
    b_values.append(b)

# Estimation des paramètres et incertitudes, ici avec les fonctions mean() et std()
a_mean = np.mean(a_values)
b_mean = np.mean(b_values)
ua = np.std(a_values)
ub = np.std(b_values)

# On peut ensuite afficher les valeurs
```